

Sintassi di base

Sintassi di base

Le regole fondamentali della sintassi Python — come "parla" il linguaggio.

Perché le regole esistono?

Ogni linguaggio, anche quelli umani come l'italiano, ha delle regole grammaticali. Se scrivi "io mangio la pizza" funziona, ma se scrivi "pizza la mangio io" il significato cambia, e in certi casi non si capisce più nulla.

Python funziona allo stesso modo: ha delle **regole di sintassi** che devi rispettare. Se le rompi, Python si ferma e ti mostra un errore. Non perché sia cattivo, ma perché non riesce a capire cosa vuoi fare.

L'indentazione: gli spazi contano!

Questa è la regola più importante e più diversa rispetto ad altri linguaggi.

In Python, gli **spazi all'inizio di una riga** non sono opzionali o decorativi: **fanno parte del codice**. Servono a dire a Python quali istruzioni stanno "dentro" ad un blocco (come un `if` o un ciclo) e quali invece stanno "fuori".

Immagina di scrivere una ricetta:

```
Se hai la farina:  
    mescola con l'acqua  
    aggiungi il sale  
prepara il forno
```

Le righe rientrate (con gli spazi davanti) appartengono alla condizione "se hai la farina". L'ultima riga invece vale sempre, indipendentemente dalla farina. Python funziona esattamente così.

```
if 5 > 3:  
    print("Cinque è maggiore di tre") # questa riga è DENTRO l'if  
print("Questa riga è FUORI dall'if")  # questa viene eseguita sempre
```

Lo standard è usare **4 spazi** per ogni livello di rientro. Non mescolare spazi e tabulazioni (il tasto Tab): causa problemi.

Se dimentichi il rientro, Python ti avvisa con un errore chiarissimo:

```
if 5 > 3:
print("Errore!") # IndentationError: expected an indented block
```

Maiuscole e minuscole

Python distingue sempre tra lettere maiuscole e minuscole. Questo si chiama essere **case-sensitive**.

Quindi `nome`, `Nome` e `NOME` sono **tre cose completamente diverse** per Python:

```
nome = "Alice"
Nome = "Bob"
print(nome) # Alice
print(Nome) # Bob
```

Fai attenzione anche a `True` e `False`: devono avere la prima lettera maiuscola. `true` o `TRUE` causano un errore.

Istruzioni su più righe

Di solito scrivi un'istruzione per riga. Se un'istruzione è molto lunga e vuoi spezzarla, puoi usare il carattere `\` alla fine della riga per dire a Python "continua alla riga successiva":

```
risultato = 10 + 20 + \
            30 + 40
print(risultato) # 100
```

Oppure, se apri una parentesi, Python capisce da solo che l'istruzione non è ancora finita:

```
numeri = (1, 2, 3,
          4, 5, 6)
```

Il punto e virgola (non è necessario)

In alcuni linguaggi, ogni istruzione deve finire con `;`. In Python **non serve**. Una riga = un'istruzione, e basta.

Potresti scrivere più istruzioni sulla stessa riga usando `;`, ma è una cattiva abitudine che rende il codice difficile da leggere:

```
# Funziona, ma è meglio evitarlo
a = 1; b = 2; print(a + b)

# Molto meglio così:
a = 1
b = 2
print(a + b)
```

Le parole riservate (keywords)

Python ha alcune parole speciali che hanno già un significato preciso nel linguaggio. Non puoi usarle come nomi di variabili.

Per esempio, `if` significa "se" per Python. Se provi a creare una variabile chiamata `if`, Python si confonde e dà errore.

Queste parole si chiamano **keyword**:

False	await	else	import	pass
None	break	except	in	raise
True	class	finally	is	return
and	continue	for	lambda	try
as	def	from	nonlocal	while
assert	del	global	not	with
async	elif	if	or	yield

Non devi memorizzarle tutte ora. Le imparerai una alla volta quando ne avrai bisogno. L'importante è sapere che esistono: se Python ti dà un errore strano su un nome di variabile, controlla che non sia una keyword.